

Team Project: Wildfire CA

Wildfire spread simulation using a cellular automaton

General description

The goal of this project is to create a wildfire simulation based on a **cellular automaton** operating on a **two-dimensional grid**. The system should make it possible to study the dynamics of fire spread under different environmental conditions and to test damage-limitation strategies such as creating firebreaks or placing barriers.

The project combines elements of **computational physics**, **object-oriented programming**, **data analysis**, and **team-based software development around a shared system**. An important part of the task is not only implementing the model itself, but also designing an architecture that allows several teams to work on different layers of the same solution.

Cellular automata are a good tool for modeling this kind of phenomenon because they rely on simple local rules while still producing complex global behavior. This gives the project a low entry barrier, while leaving plenty of room for extensions, experiments, and further development.

Project objective

You are expected to design and implement a system that:

- simulates wildfire spread on a 2D grid,
- allows visualization of the system's evolution over time,
- measures basic properties of the fire dynamics,
- supports testing different terrain protection strategies,
- enables analysis of results and comparison of model variants.

The project should be developed as **one shared system**, built by several teams collaborating through clearly defined interfaces.

Core requirements (MVP)

To be considered complete in its basic version, the system should provide the following functionality.

1. Simulation space

The simulation should run on a **two-dimensional grid** representing a piece of terrain.

2. Cell states

Each cell should take at least one of the following states:

- **ground / empty cell,**
- **vegetation / fuel,**
- **burning,**
- **burned.**

Equivalent naming is acceptable as long as the meaning of the states is clearly documented.

3. Propagation mechanics

At least one neighborhood type must be implemented:

- **von Neumann,**
- or **Moore.**

The fire should spread to neighboring flammable cells according to clearly defined rules. The model may be:

- deterministic,
- or probabilistic.

In the basic version, it is enough for a burning cell to ignite neighboring fuel cells, and after a specified amount of time to transition into the burned state.

4. Initialization and configuration

The system should allow:

- setting the initial ignition point,
- creating a simple initial map,
- changing the main model parameters.

Support for loading configuration from a file is welcome.

5. Visualization

A basic visualization of the grid state over time should be provided. A simple and readable representation using colors or symbols is sufficient.

The visualization should allow:

- starting the simulation,
- pausing or resetting it,
- observing successive time steps.

6. Measurement and metrics

The system should collect at least the following metrics:

- percentage of burned area,
- number of currently burning cells,

- total wildfire duration measured in simulation steps,
 - information about whether the fire reached a chosen map boundary or spread across the entire studied area.
-

Project extensions

Once the core version is complete, the system may be extended with additional model and system features.

1. Wind modeling

A **wind vector** may be introduced, defined by direction and strength, to modify ignition probability asymmetrically. Fire may then spread faster downwind and slower upwind.

2. Heterogeneous terrain

Different terrain or land-cover types may be included, for example:

- dry grass,
- forest,
- wet terrain,
- rocks,
- water.

Each terrain type may have different properties such as:

- ignition susceptibility,
- burning duration,
- ability to block fire spread.

3. Randomness and statistical experiments

Random ignition or other stochastic effects may be added to the model. In that case, it is recommended to support:

- an adjustable **random seed**,
- reproducible experiments,
- statistical analysis over many runs.

4. Firebreaks and interventions

The system may support:

- manual creation of firebreaks,
- removal of fuel from selected cells,

- placement of non-flammable barriers,
- interventions performed either before the fire starts or during the simulation.

5. Defense strategy and AI

As an extension, you may implement an algorithm that chooses defensive actions. This may be:

- a simple heuristic,
- a rule-based strategy,
- or an AI agent using a prepared API.

It is important that such a strategy operates under a **limited action budget**, for example:

- a limited number of interventions,
 - a limited number of cells that can be protected,
 - limited reaction time.
-

Work organization and team structure

The project should be carried out by **three cooperating teams** developing one shared system. This requires early agreement on interfaces between modules and on the format of exchanged data.

Team 1: Simulation engine (Core & Physics)

This team is responsible for the model core and the cellular automaton logic.

Responsibilities

- representation of the grid and cells,
- implementation of states and transition rules,
- time-step updates,
- scenario initialization,
- computation of basic metrics,
- preparation of an interface for communication with other modules.

Technical requirement

The engine should run in **headless** mode, meaning without any graphical layer. This makes it possible to:

- test the logic independently of the GUI,
 - run many simulations in the background,
 - use the engine from the analytics module and from a possible AI agent.
-

Team 2: Visualization and interface (GUI / API)

This team is responsible for the presentation layer and for user interaction with the system.

Responsibilities

- visualization of the grid state,
- creation of a basic user interface,
- starting, stopping, and resetting simulations,
- displaying metrics,
- enabling scenario loading and parameter changes,
- preparing the communication layer with the engine.

The interface does not need to be sophisticated. The key requirement is that it should be clear and convenient for running experiments.

Team 3: Strategies, data analysis, and extensions

This team is responsible for using the simulator as an experimental environment.

Responsibilities

- designing research scenarios,
- collecting simulation results,
- preparing plots and summaries,
- comparing different protection strategies,
- implementing simple defense methods,
- optionally: integrating an AI agent.

Example tasks

- studying how vegetation density affects the risk of a global wildfire,
 - comparing different neighborhood types,
 - analyzing the effect of wind,
 - comparing strategies: no defense, random defense, heuristic defense, AI-assisted defense.
-

Engineering requirements

1. API-first design

Before implementation begins, the teams should jointly define:

- the data model,

- the representation of simulation state,
- the set of basic operations provided by the engine,
- the format used for communication between modules.

The GUI and analytics teams should not directly manipulate the internal variables of the engine module. Communication should happen through clearly defined interfaces.

2. Shared configuration format

It is recommended to adopt a shared format for scenario and parameter descriptions, for example:

- JSON,
- YAML,
- or another readable text-based format.

Such a file may contain:

- grid size,
- neighborhood type,
- ignition parameters,
- initial fire position,
- obstacle placement,
- environmental settings.

3. Git and workflow organization

Development should be managed in a Git repository using a sensible workflow.

Recommended rules:

- each new feature should be developed on a separate branch,
- changes should be merged into the main branch through a **pull request**,
- each pull request should be reviewed by a person from another team,
- the repository should include an up-to-date README with setup and run instructions.

4. Testability and reproducibility

The system should be designed so that it is possible to:

- run simulations without the graphical interface,
 - reproduce experiments from a saved configuration,
 - in stochastic versions, repeat results for a fixed random seed.
-

Example research questions

As part of the project, it is worth formulating and investigating a few specific questions, for example:

- How does the choice of neighborhood affect wildfire dynamics?
 - How do the results change when randomness is introduced?
 - What is the effect of wind direction and strength?
 - Can a small number of well-placed barriers significantly reduce damage?
 - Which protection strategy works best under a limited action budget?
 - Does an AI-based strategy outperform a simple heuristic?
-

Expected final deliverables

At the end of the project, the group should present:

- a working wildfire simulation system,
- integrated modules from all teams,
- a short presentation of the system architecture,
- a set of conducted experiments,
- interpretation of the results,
- brief technical documentation and setup instructions.

A short report is also welcome, including:

- a description of the model,
 - a description of the assumptions,
 - a list of implemented features,
 - comparison of variants,
 - conclusions from the experiments.
-

Assessment criteria

Project evaluation may take into account the following aspects:

- correctness and consistency of the model,
- code quality and software architecture,
- level of integration between modules,
- clarity of the interface and presentation of results,

- quality of experiments and data analysis,
 - organization of team collaboration,
 - scope and quality of implemented extensions.
-

Summary

The **Wildfire CA** project is intended to combine a simple but interesting physical model with engineering practice and data analysis. Its main advantage is that it allows you to begin with a small, well-defined core and then extend the system in a modular way. This makes it a strong topic for team-based work, interface design, and collaborative software development.

If planned well, the project can become not only an effective visual simulation, but also a полноценный environment for numerical experiments and for comparing decision strategies in a complex dynamic system.